

Resumen de Bibliografía

Database System Concepts — Silberschatz, Korth, Sudarshan (2019)

Capítulos 12 (12.1–12.3, 12.6), 13 (13.1–13.4), 14 (14.1–14.5), 15, 16 (16.1–16.4)

Capítulo 12 — Sistemas de Almacenamiento Físico

12.1 — Panorama de los Medios de Almacenamiento Físico

Los medios de almacenamiento se clasifican por **velocidad de acceso**, **costo por unidad de dato** y **confiabilidad**. La jerarquía de almacenamiento (de más rápido y caro a más lento y barato) es:

1. **Caché:** más rápida y costosa. Administrada por hardware. Muy pequeña.
2. **Memoria principal:** volátil (se pierde ante cortes de energía). Decenas a miles de GB. Instrucciones de CPU operan sobre ella.
3. **Memoria flash:** no volátil. Menor costo/byte que memoria principal pero mayor que discos magnéticos. Base de las SSDs. Interfaz orientada a bloques (típicamente 512 bytes a 8 KB).
4. **Disco magnético (HDD):** no volátil. Almacenamiento principal a largo plazo. Capacidades de 500 GB a 14 TB. Más barato que SSD pero más lento.
5. **Almacenamiento óptico (DVD/Blu-ray):** 27–128 GB. No apto para datos activos de BD.
6. **Cinta magnética:** acceso secuencial. 1–12 TB. Muy barata. Usada para backup y archivo.

Clasificación según volatilidad:

- **Almacenamiento primario:** caché y memoria principal (volátiles).
- **Almacenamiento secundario (online):** flash y disco magnético (no volátiles).
- **Almacenamiento terciario (offline):** cintas y jukeboxes ópticos.

12.2 — Interfaces de Almacenamiento

Los discos se conectan al sistema mediante interfaces de alta velocidad:

- **SATA (Serial ATA):** hasta 600 MB/s (SATA-3). Uso general.
- **SAS (Serial Attached SCSI):** hasta 12 Gbps. Uso en servidores.
- **NVMe (Non-Volatile Memory Express):** interfaz lógica optimizada para SSDs, sobre PCIe. Mayor rendimiento.

Arquitecturas de almacenamiento remoto:

- **SAN (Storage Area Network):** red de alta velocidad conectando discos a servidores. Usa RAID internamente. Presenta vista lógica de disco grande y confiable. Protocolos: iSCSI, Fiber Channel, InfiniBand.
- **NAS (Network Attached Storage):** similar a SAN pero provee interfaz de sistema de archivos (NFS, CIFS).
- **Cloud storage:** acceso vía API. Latencia alta (decenas a cientos de ms). No ideal como almacenamiento subyacente para BD activas.

12.3 — Discos Magnéticos

12.3.1 Características Físicas

Un disco magnético consta de **platos** circulares rígidos cubiertos de material magnético, montados sobre un **eje (spindle)** que gira a velocidad constante (5400–15000 RPM). Cada superficie se divide en:

- **Pistas (tracks):** círculos concéntricos en la superficie del plato.

- **Sectores:** subdivisiones de cada pista. Unidad mínima de lectura/escritura (típicamente 512 bytes).
- **Cilindro:** conjunto de las i -ésimas pistas de todos los platos.

Una **cabeza de lectura/escritura** (read-write head) se posiciona sobre la superficie del plato, montada en un **brazo de disco (disk arm)**. Todas las cabezas se mueven juntas (ensamblaje de brazo). La cabeza “flota” a micrones de la superficie.

El **controlador de disco** acepta comandos de alto nivel, mueve el brazo, lee/escribe datos, agrega **checksums** a cada sector para detectar corrupción, y realiza **remapeo de sectores defectuosos**.

12.3.2 Métricas de Rendimiento

- **Tiempo de acceso** = tiempo de búsqueda (seek time) + latencia rotacional.
 - **Seek time:** tiempo para mover el brazo a la pista correcta. Promedio: 4–10 ms.
 - **Latencia rotacional:** espera a que el sector pase bajo la cabeza. Promedio: medio giro (2–5.5 ms según RPM).
- **Tasa de transferencia:** velocidad de lectura/escritura una vez posicionado. Máximo: 50–200 MB/s; menor en pistas internas.
- **Bloque de disco:** unidad lógica de transferencia. Típicamente 4–16 KB.
- **IOPS (I/O operations per second):** cantidad de accesos aleatorios por segundo. Discos actuales: 50–200 IOPS.
- **MTTF (Mean Time To Failure):** tiempo medio hasta fallo. Valores reportados: 500,000–1,200,000 horas (vida útil real $\approx$ 5 años).

Patrones de acceso:

- **Secuencial:** bloques consecutivos, mínimo seek $\rightarrow$ máxima tasa de transferencia.
- **Aleatorio:** cada acceso requiere seek $\rightarrow$ rendimiento mucho menor.

12.6 — Acceso a Bloques de Disco

Técnicas para mejorar la velocidad de acceso a bloques:

1. **Buffering:** bloques leídos se almacenan temporalmente en un buffer en memoria para satisfacer futuras solicitudes.
2. **Read-ahead (lectura anticipada):** se leen bloques consecutivos de la misma pista anticipadamente, aunque no hayan sido solicitados. Efectivo para acceso secuencial; poco útil para acceso aleatorio.
3. **Disk-arm scheduling:** reordenar solicitudes de acceso para minimizar movimiento del brazo. El **algoritmo del elevador** mueve el brazo en una dirección atendiendo solicitudes, luego invierte la dirección (como un ascensor).
4. **Organización de archivos:** organizar bloques en disco según el patrón de acceso esperado. Bloques consecutivos se numeran lógicamente como adyacentes. Archivos se asignan en **extents** (grupos de bloques consecutivos). La **fragmentación** se combate con desfragmentación o backup/restore.
5. **Buffers de escritura no volátiles (NVRAM):** cuando el sistema solicita escribir un bloque, el controlador lo escribe en un buffer NVRAM y reporta éxito inmediato. Luego escribe a disco optimizando movimiento del brazo. Ante crash, las escrituras pendientes se recuperan del NVRAM.

Capítulo 13 — Estructuras de Almacenamiento de Datos

13.1 — Arquitectura de Almacenamiento de BD

Los datos persistentes residen en almacenamiento no volátil (disco magnético o SSD). Los discos son dispositivos **orientados a bloques**: datos se leen/escriben en unidades de bloques. Las bases de datos trabajan con **registros**, que son mucho más pequeños que un bloque.

La mayoría de BD usan **archivos del sistema operativo** como capa intermedia para almacenar registros, pero deben ser conscientes de los bloques para eficiencia y recuperación.

Decisiones clave:

- Cómo representar registros individuales dentro de bloques (Sección 13.2).
- Cómo organizar registros en archivos (Sección 13.3).
- Cómo organizar metadatos en el diccionario de datos (Sección 13.4).

Bases de datos que caben en memoria principal se llaman **main-memory databases**; almacenamiento orientado a columnas (**column-oriented storage**) es eficiente para consultas analíticas.

13.2 — Organización de Archivos

Una BD se mapea a varios archivos del SO. Un **archivo** es una secuencia lógica de registros mapeados a bloques de disco. Los **bloques** son la unidad de asignación y transferencia (típicamente 4–8 KB). Asumimos que **ningún registro es mayor que un bloque** y que cada registro cabe **enteramente en un solo bloque**.

13.2.1 Registros de Longitud Fija

Cada registro ocupa exactamente el mismo número de bytes. Los registros se almacenan consecutivamente en el bloque. Problemas:

- Si el tamaño del registro no divide exactamente al tamaño del bloque, algunos registros cruzarían límites de bloque → se desperdician bytes al final de cada bloque.
- **Borrado**: el espacio liberado debe reutilizarse. Opciones: mover el último registro al hueco, o usar una **lista libre (free list)** con un puntero en el **encabezado del archivo (file header)** al primer registro borrado, y cada registro borrado apunta al siguiente.

13.2.2 Registros de Longitud Variable

Surgen por campos de longitud variable (strings), campos repetidos, o múltiples tipos de registro en un archivo. Dos problemas:

1. Representar un registro individual para extraer atributos fácilmente.
2. Almacenar registros de longitud variable en un bloque.

Representación de un registro: parte inicial fija con información de cada atributo. Atributos de longitud fija se almacenan directamente. Atributos de longitud variable se representan como un par (offset, longitud). Los valores variables van después de la parte fija. Se usa un **null bitmap** para indicar atributos nulos.

Estructura de página con ranuras (slotted-page structure): método estándar para organizar registros variables en un bloque. El encabezado del bloque contiene:

- Número de entradas de registro.
- Fin del espacio libre.
- Un arreglo con ubicación y tamaño de cada registro.

Los registros se asignan contiguamente desde el **final del bloque**. El espacio libre está entre el arreglo del encabezado y los registros. Los punteros externos apuntan a la entrada del encabezado, no directamente al registro, permitiendo mover registros dentro del bloque sin invalidar punteros.

13.2.3 Almacenamiento de Objetos Grandes

Objetos como imágenes, audio o video pueden ser mucho más grandes que un bloque. SQL soporta tipos **blob** y **clob**. Se almacenan separados del registro, con un puntero lógico en el registro. Pueden usarse organizaciones de archivos B⁺-tree para acceso eficiente a partes del objeto. Muchas aplicaciones almacenan objetos grandes fuera de la BD (en el sistema de archivos), guardando solo la ruta como atributo.

13.3 — Organización de Registros en Archivos

Formas de organizar registros dentro de archivos:

13.3.1 Organización Heap (Montículo)

Los registros se colocan en cualquier lugar del archivo donde haya espacio. No hay ordenamiento. Se usa un **free-space map**: arreglo con una entrada por bloque indicando la fracción de espacio libre (típicamente 1 byte por bloque en PostgreSQL). Un free-space map de segundo nivel almacena el máximo de cada grupo de entradas del mapa principal para acelerar búsquedas. El mapa se escribe periódicamente a disco.

13.3.2 Organización Secuencial

Un **archivo secuencial** almacena registros en orden según una **clave de búsqueda (search key)**: cualquier atributo o conjunto de atributos. Los registros se encadenan con punteros en orden de clave. Permite lectura en orden y procesamiento secuencial eficiente.

Inserciones: se busca el registro anterior en orden de clave; si hay espacio en el mismo bloque, se inserta allí; sino, se usa un **bloque de desbordamiento (overflow block)**. La correspondencia entre orden lógico y físico se degrada con el tiempo, requiriendo **reorganización** periódica.

13.3.3 Organización Multitable Clustering

Normalmente cada relación va en su propio archivo. En **multitable clustering**, registros de dos o más relaciones relacionadas se almacenan en el mismo bloque, agrupados por una **clave de cluster** (ej: `dept_name`).

Ventaja: joins sobre la clave de cluster son muy eficientes (registros relacionados en el mismo bloque). Desventaja: consultas sobre una sola relación requieren más accesos a bloques. Oracle lo soporta con `CREATE CLUSTER`.

13.3.4 Particionamiento de Tablas

Las relaciones pueden particionarse en subrelaciones más pequeñas basándose en un atributo (ej: particionar *transaction* por año). Beneficios:

- Consultas filtradas por el atributo de partición solo acceden a la subrelación relevante.
- Reduce overhead de operaciones como búsqueda de espacio libre.
- Permite almacenar particiones frías en disco magnético y calientes en SSD.

13.4 — Almacenamiento del Diccionario de Datos

El **diccionario de datos** (o **catálogo del sistema**) almacena **metadatos**: datos sobre los datos. Información almacenada:

- Nombres de relaciones y sus atributos (dominios, longitudes).
- Nombres y definiciones de vistas.
- Restricciones de integridad (claves, etc.).
- Información de usuarios, contraseñas, autorizaciones.
- Organización de almacenamiento (heap, secuencial, hash, etc.) y ubicación de cada relación.
- Información de índices: nombre, relación, atributos indexados, tipo.
- Estadísticas: número de tuplas, valores distintos por atributo.

Los metadatos se almacenan como relaciones dentro de la propia BD. Un esquema ilustrativo incluye relaciones como *Relation_metadata*, *Attribute_metadata*, *Index_metadata*, *View_metadata* y *User_metadata*. Se almacena en forma desnormalizada para acceso rápido. Al ser accedido frecuentemente, se carga en estructuras in-memory al inicio de la BD.

Capítulo 14 — Indexación

14.1 — Conceptos Básicos

Un **índice** es una estructura adicional asociada a un archivo que permite acceso rápido a registros sin escanear toda la relación. Funciona como el índice de un libro: almacena claves en orden con punteros a los registros.

Dos tipos básicos de índices:

- **Índices ordenados**: basados en ordenamiento de valores.
- **Índices hash**: basados en distribución uniforme de valores mediante una función hash.

Factores de evaluación de un índice:

- **Tipos de acceso**: búsqueda por igualdad, búsqueda por rango.
- **Tiempo de acceso**.
- **Tiempo de inserción y eliminación** (incluye actualizar el índice).
- **Overhead de espacio**.

Una **clave de búsqueda** (**search key**) es un atributo o conjunto de atributos usado para buscar registros en un archivo. No es lo mismo que clave primaria o candidata.

14.2 — Índices Ordenados

Un **índice ordenado** almacena claves de búsqueda en orden y asocia con cada clave los registros que la contienen.

Índice de clustering vs. índice secundario

- **Índice de clustering (primario)**: la clave del índice define el orden secuencial del archivo. Se aplica a archivos **index-sequential**.
- **Índice no clustering (secundario)**: la clave especifica un orden diferente al orden físico del archivo.

14.2.1 Índices Densos y Dispersos

Una **entrada de índice** (index record) contiene un valor de clave y puntero(s) a registros.

- **Índice denso:** una entrada por cada valor de clave en el archivo.
 - En clustering denso: cada entrada apunta al primer registro con ese valor de clave.
 - En no clustering denso: cada entrada almacena lista de punteros a todos los registros con esa clave.
- **Índice disperso (sparse):** entradas solo para algunos valores de clave. Solo para índices de clustering (archivo ordenado por la clave). Localización: buscar la mayor clave $\leq$ al valor buscado y recorrer secuencialmente desde ahí.

Trade-off: denso es más rápido para búsqueda; disperso requiere menos espacio y menos mantenimiento. Compromiso práctico: una entrada dispersa por bloque.

14.2.2 Índices Multinivel

Cuando un índice es demasiado grande para caber en memoria, la búsqueda binaria sobre él requiere muchos accesos aleatorios a disco ($\lceil \log_2(b) \rceil$ bloques para b bloques de índice).

Solución: tratar al índice como un archivo secuencial y construir un **índice disperso externo (outer index)** sobre él. El índice original pasa a ser el **inner index**. Se puede repetir para más niveles.

Ejemplo: con 10,000 bloques de inner index, el outer index ocupa solo 100 bloques. Si el outer index cabe en memoria, una búsqueda requiere solo 1 lectura de bloque (vs. 14 con búsqueda binaria).

14.2.3 Actualización de Índices

Todo índice debe actualizarse ante inserciones y eliminaciones.

Inserción en índice denso: si la clave no existe, se inserta nueva entrada. Si existe, se agrega puntero o se coloca el registro después del primero con esa clave.

Inserción en índice disperso: si se crea un nuevo bloque, se inserta entrada con la primera clave del nuevo bloque. Si el nuevo registro tiene la menor clave del bloque, se actualiza la entrada existente.

Eliminación: en denso, se elimina la entrada si era el único registro con esa clave; sino se actualiza el puntero. En disperso, se reemplaza con la siguiente clave o se elimina la entrada.

14.2.4 Índices Secundarios

Los índices secundarios **deben ser densos**: como el archivo no está ordenado por la clave secundaria, no se puede usar recorrido secuencial para encontrar registros intermedios.

Para claves no candidatas (nonunique), los punteros del índice secundario apuntan a **buckets** (bloques de punteros) que a su vez apuntan a los registros. Esto agrega un nivel de indirección.

Ventaja: mejoran consultas sobre claves distintas a la de clustering. Desventaja: overhead significativo en actualizaciones (cada modificación del archivo debe actualizar **todos** los índices). El acceso secuencial por clave secundaria es muy costoso (cada registro puede estar en un bloque diferente).

14.2.5 Índices sobre Múltiples Claves

Una **clave de búsqueda compuesta** contiene más de un atributo. El ordenamiento es **lexicográfico**: $(a_1, a_2) < (b_1, b_2)$ si $a_1 < b_1$, o si $a_1 = b_1$ y $a_2 < b_2$. Útil para consultas que combinan condiciones sobre múltiples atributos.

14.3 — Archivos de Índice B⁺-Tree

El principal problema de la organización index-sequential es la **degradación del rendimiento** a medida que el archivo crece (desbordamientos, reorganizaciones costosas).

El **B⁺-tree** es la estructura de índice más usada en BD. Es un **árbol balanceado**: todos los caminos de la raíz a las hojas tienen la misma longitud.

14.3.1 Estructura del B⁺-Tree

Cada nodo contiene hasta $n - 1$ claves $K_1, K_2, \dots, K_{n-1}$ y n punteros $P_1, P_2, \dots, P_n$. Las claves están ordenadas: $K_i < K_j$ si $i < j$.

Nodos hoja:

- Para $i = 1, \dots, n - 1$: puntero P_i apunta al registro con clave K_i .
- P_n apunta al siguiente nodo hoja (encadenamiento para recorrido secuencial).
- Cada hoja tiene entre $\lceil (n - 1)/2 \rceil$ y $n - 1$ valores.
- Si se usa como índice denso, todo valor de clave aparece en alguna hoja.

Nodos internos (no hoja):

- Forman un índice disperso multinivel sobre las hojas.
- Puntero P_i apunta al subárbol con claves $\geq K_{i-1}$ y $< K_i$.
- Cada nodo interno tiene entre $\lceil n/2 \rceil$ y n punteros (hijos).
- El **fanout** es el número de punteros en un nodo.
- La raíz puede tener mínimo 2 punteros.

Profundidad del árbol para N registros: $\leq \lceil \log_{\lceil n/2 \rceil}(N) \rceil$.

Ejemplo práctico: con $n = 100$ y 1,000,000 claves, la búsqueda requiere solo $\lceil \log_{50}(1,000,000) \rceil = 4$ accesos a nodos. La raíz suele estar en el buffer $\rightarrow$ 3 o menos lecturas de disco.

14.3.2 Consultas en B⁺-Trees

Búsqueda puntual (find(v)): desde la raíz, en cada nodo encontrar el menor $K_i \geq v$ y seguir el puntero correspondiente. Repetir hasta llegar a una hoja. Costo: proporcional a la altura del árbol.

Consultas de rango (findRange(lb, ub)): encontrar la hoja del límite inferior, luego recorrer hojas consecutivas recolectando punteros hasta superar el límite superior.

Para claves no únicas, se crea una clave compuesta (a_i, A_p) donde A_p es la clave primaria, garantizando unicidad.

14.3.3 Actualizaciones en B⁺-Trees

Inserción:

1. Buscar la hoja donde debe ir la nueva clave.
2. Si hay espacio, insertar en orden.
3. Si la hoja está llena, **split**: dividir en dos nodos, los primeros $\lceil n/2 \rceil$ quedan en el original, el resto en el nuevo. La menor clave del nuevo nodo se inserta en el padre.
4. Si el padre se llena, se repite el split recursivamente. Si la raíz se divide, se crea nueva raíz (el árbol crece en profundidad).
5. **Diferencia entre split de hoja y de nodo interno**: en hojas, la clave “promovida” se copia al padre (permanece en la hoja). En nodos internos, la clave se **mueve** al padre (no queda en ninguno de los nodos divididos).

Eliminación:

1. Buscar y eliminar la entrada de la hoja.
2. Si la hoja queda con menos de $\lceil (n-1)/2 \rceil$ valores (**underfull**):
 - Intentar **redistribuir** con un hermano (tomar prestada una entrada).
 - Si no es posible, **fusionar (coalesce)** con un hermano y eliminar la entrada separadora del padre.
3. Las fusiones pueden propagarse recursivamente hasta la raíz. Si la raíz queda con un solo hijo, ese hijo se convierte en la nueva raíz.

14.3.4 Complejidad de Actualizaciones

Costo en I/O de inserción y eliminación: proporcional a $\log_{\lceil n/2 \rceil}(N)$ (altura del árbol). En la práctica, con fanout de 100, solo 1–2 lecturas de disco son necesarias. La probabilidad de split es muy baja (1 de cada 100 inserciones con fanout 100). Los nodos tienden a estar 2/3 llenos en promedio.

14.3.5 Claves de Búsqueda No Únicas

Para manejar duplicados, la mayoría de implementaciones usan el enfoque de **clave única compuesta**: se agrega un atributo **uniquifier** (típicamente la clave primaria o un record-id) a la clave de búsqueda. Esto simplifica inserción y eliminación, manteniendo la complejidad logarítmica.

14.4 — Extensiones de B⁺-Tree*14.4.1 Organización de Archivo B⁺-Tree*

En una **B⁺-tree file organization**, los nodos hoja almacenan **registros completos** en lugar de punteros. Resuelve la degradación de archivos index-sequential. Cada bloque tiene al menos la mitad de registros llenos.

Para mejorar la utilización de espacio, durante splits se involucran nodos hermanos: redistribuir entre el nodo y sus adyacentes antes de crear un nuevo nodo. Esto garantiza $\lfloor 2n/3 \rfloor$ entradas mínimas por nodo.

14.4.2 Índices Secundarios y Reubicación de Registros

Problema: si la organización B⁺-tree mueve registros (por splits), todos los índices secundarios deben actualizarse.

Solución: los índices secundarios almacenan **valores de clave primaria** en lugar de punteros directos. Localizar un registro requiere dos pasos: buscar la clave primaria en el índice secundario, luego usar el índice primario. Reduce costo de actualización a cambio de mayor costo de acceso.

14.4.3 Indexación de Strings

Problemas: strings largos reducen el fanout. Soluciones:

- Nodos con fanout variable (según longitud de claves).
- **Compresión de prefijos**: en nodos internos, almacenar solo el prefijo necesario para distinguir entre subárboles.

14.4.4 Carga Masiva (Bulk Loading)

Insertar registros uno a uno en un B^+ -tree grande no clustering es muy costoso (acceso aleatorio por cada inserción).

Bulk loading: ordenar las entradas del índice por clave, luego insertarlas en orden. Beneficios:

- Cada nodo hoja se escribe solo una vez.
- Las I/O son mayormente secuenciales.

Construcción bottom-up: partir las entradas ordenadas en bloques (hojas), luego construir niveles superiores con los valores mínimos de cada bloque. Mucho más eficiente que inserción individual.

14.4.5 Archivos de Índice B-Tree

El **B-tree** (sin “+”) elimina la redundancia: cada valor de clave aparece **una sola vez** en todo el árbol. Los nodos internos incluyen punteros a registros (B_i) además de punteros a hijos (P_i). Ventaja teórica: menos nodos. Desventajas prácticas:

- Menor fanout en nodos internos $\rightarrow$ mayor profundidad.
- Eliminación más compleja.
- Implementación más complicada.
- Ventaja de espacio marginal.

En la práctica, **casi todas las BD usan B^+ -trees**, aunque muchas los llaman “B-trees”.

14.5 — Índices Hash

Hashing es una técnica para construir índices en memoria principal o en disco. Un **bucket** es una unidad de almacenamiento (lista enlazada en memoria, o bloque(s) de disco).

Sea K el conjunto de claves y B el conjunto de direcciones de buckets. Una **función hash** $h : K \rightarrow B$ mapea cada clave a un bucket. La técnica más usada es **overflow chaining** (closed addressing): cada bucket tiene una lista enlazada de entradas, y los buckets que desbordan se encadenan con **overflow buckets**.

Operaciones:

- **Búsqueda:** calcular $h(K_i)$, buscar en el bucket (y sus overflow buckets).
- **Inserción:** calcular $h(K_i)$, insertar en el bucket; si está lleno, crear overflow bucket.
- **Eliminación:** calcular $h(K_i)$, buscar y eliminar del bucket/lista.

Limitación principal: los índices hash **no soportan consultas de rango**. Solo sirven para búsquedas por igualdad.

Bucket overflow puede ocurrir por:

- Número insuficiente de buckets.
- **Skew** (distribución no uniforme): múltiples registros con la misma clave, o función hash con distribución no uniforme.

Para reducir overflow, se asignan $(n_r/f_r) \times (1 + d)$ buckets, donde n_r = número de registros, f_r = registros por bucket, y d es un factor de holgura ($\approx 0,2$).

Hashing estático vs. dinámico:

- **Estático:** número fijo de buckets, definido al crear el índice. Si los datos crecen mucho, se degradan las búsquedas. Reconstruir el índice es costoso.
- **Dinámico:** permite incrementar el número de buckets gradualmente (*linear hashing, extendable hashing*).

Capítulo 15 — Procesamiento de Consultas

15.1 — Panorama General

El procesamiento de consultas comprende las actividades para extraer datos de una BD. Pasos:

1. **Parsing y traducción:** verificar sintaxis y traducir SQL a álgebra relacional.
2. **Optimización:** elegir el plan de evaluación de menor costo entre los equivalentes.
3. **Evaluación:** ejecutar el plan elegido y devolver resultados.

Un **plan de evaluación** define exactamente qué algoritmo usar para cada operación y cómo coordinar la ejecución. La diferencia de costo entre un plan bueno y uno malo puede ser de **varios órdenes de magnitud**.

Tres pasos del optimizador: (1) generar expresiones lógicamente equivalentes, (2) anotar cada expresión con algoritmos alternativos, (3) estimar costos y elegir el menor.

15.2 — Medidas de Costo de una Consulta

El costo se mide en términos de **transferencias de bloques** (t_T) y **búsquedas de disco** (t_S):

$$\text{Costo} = b \times t_T + S \times t_S$$

donde b = número de bloques transferidos y S = número de seeks. Típicamente $t_S \gg t_T$ (ej: $t_S = 4$ ms, $t_T = 0,1$ ms).

M = número de bloques de memoria disponibles para el buffer. Más memoria $\rightarrow$ menos I/O. En peor caso, $M = 3$ (mínimo para muchos algoritmos).

Se ignora el costo de escribir la salida final (es igual para todos los planes).

15.3 — Operación de Selección

Algoritmos de selección

Algo.	Nombre	Descripción y costo
A1	<i>Linear search</i>	Escaneo completo. Costo: $b_r \times t_T + t_S$. Para igualdad sobre clave: promedio $\frac{b_r}{2} \times t_T + t_S$.
A2	<i>Clustering index, igualdad</i>	Índice primario/clustering. Costo: $(h_i + 1) \times (t_T + t_S)$.
A3	<i>Clustering index, igualdad (múltiples)</i>	Clave no candidata: $(h_i + \lceil b/V(A, r) \rceil) \times t_T + 2 \times t_S$.
A4	<i>Secondary index, igualdad</i>	Clave candidata: $(h_i + 1) \times (t_T + t_S)$. No candidata: $(h_i + n) \times (t_T + t_S)$ donde n = registros que matchean.
A5	<i>Clustering index, comparación</i>	$\sigma_{A \geq v}$: un seek + escanear desde el punto. Costo $\approx h_i \times (t_T + t_S) + \frac{b_r}{2} \times t_T$.
A6	<i>Secondary index, comparación</i>	Costoso: cada registro puede estar en un bloque diferente.
A7	<i>Conjunción con un índice</i>	Usar índice sobre el atributo más selectivo, luego verificar el resto en memoria.
A8	<i>Conjunción con índice compuesto</i>	Si existe índice compuesto sobre los atributos de la conjunción.
A9	<i>Conjunción por intersección de identificadores</i>	Obtener conjuntos de punteros de cada índice, interseccionarlos.
A10	<i>Disyunción por unión de identificadores</i>	Si todos los términos tienen índice: unión de conjuntos de punteros. Si alguno no tiene índice $\rightarrow$ linear search.

Nota: h_i = altura del índice (B^+ -tree). Para B^+ -tree típico, $h_i \leq 4$.

15.4 — Ordenamiento (External Sort-Merge)

Relaciones que no caben en memoria se ordenan con **external sort-merge**:

1. **Generación de runs:** leer M bloques a memoria, ordenar in-memory, escribir como un *run* ordenado. Repetir hasta procesar toda la relación. Se generan $\lceil b_r/M \rceil$ runs iniciales.
2. **Merge de N vías:** fusionar $N = M - 1$ runs a la vez (1 bloque de buffer por run + 1 bloque de salida). Si hay más de N runs, se hacen múltiples pasadas de merge.

Número de pasadas de merge: $\lceil \log_{M-1}(b_r/M) \rceil$.

Costo total:

- Transferencias de bloques: $b_r \times (2\lceil \log_{M-1}(b_r/M) \rceil + 1)$
- Seeks: $2\lceil b_r/M \rceil + \lceil b_r/b_b \rceil \times (2\lceil \log_{M-1}(b_r/M) \rceil - 1)$, donde b_b = bloques de buffer por run.

15.5 — Operación de Junta (Join)

15.5.1 Nested-Loop Join

Para cada tupla de r (outer), escanear **toda** s (inner) buscando matches.

Costo (peor caso): $n_r \times b_s + b_r$ transferencias, $n_r + b_r$ seeks.

Conviene que la relación **más chica** sea la outer.

15.5.2 Block Nested-Loop Join

Para cada **bloque** de r , escanear todos los bloques de s .

Costo (peor caso): $b_r \times b_s + b_r$ transferencias, $2 \times b_r$ seeks.

Optimización con M bloques: usar $M - 2$ bloques para r , 1 para s , 1 para salida:

$$\lceil b_r / (M - 2) \rceil \times b_s + b_r \text{ transferencias, } 2 \lceil b_r / (M - 2) \rceil \text{ seeks}$$

15.5.3 Indexed Nested-Loop Join

Si existe índice sobre la clave de junta de la relación **inner**, para cada tupla de outer se busca en el índice.

Costo: $b_r \times (t_T + t_S) + n_r \times c$, donde c = costo de una búsqueda por índice + recuperación de registros. Con B⁺-tree: $c = (h_i + 1) \times (t_T + t_S)$ para clave candidata.

15.5.4 Merge Join (Sort-Merge Join)

Ambas relaciones deben estar **ordenadas** por los atributos de junta. Se recorren en paralelo, matcheando grupos de tuplas con el mismo valor.

Costo (si ya ordenadas): $b_r + b_s$ transferencias, $\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil$ seeks.

Si no están ordenadas, sumar costo de ordenarlas con external sort-merge.

Hybrid merge join: variante que combina una relación ordenada con un índice B⁺-tree secundario de la otra, evitando ordenar ambas.

15.5.5 Hash Join

Usa una función hash h sobre los atributos de junta para **particionar** ambas relaciones en n_h particiones. Solo se comparan tuplas de particiones correspondientes (r_i con s_i).

Fases:

1. **Particionamiento:** aplicar h a cada tupla de r y s , escribir en particiones.
2. **Build & Probe:** para cada par (s_i, r_i) : construir índice hash in-memory sobre s_i (*build input*), luego recorrer r_i (*probe input*) buscando matches.

La relación **más chica** debe ser la build input. s cabe sin particionamiento recursivo si $M > \sqrt{b_s}$ (aprox.).

Costo (sin particionamiento recursivo):

- Transferencias: $3(b_r + b_s) + 4n_h$
- Seeks: $2(\lceil b_r / b_b \rceil + \lceil b_s / b_b \rceil) + 2n_h$

Particionamiento recursivo: si $n_h > M$, se re-particiona con otra función hash. Costo adicional: factor $\lceil \log_{\lfloor M/b_b \rfloor - 1} (b_s / M) \rceil$.

Hash-table overflow: ocurre por skew o demasiados duplicados. Manejo: *overflow resolution* (re-particionar al detectar) u *overflow avoidance* (particionar más finamente). El *fudge factor* ($\approx 1,2$) incrementa n_h para compensar skew.

Hybrid hash join: si la memoria es grande, la primera partición de s queda en memoria (no se escribe a disco). Ahorra un ciclo de I/O para esa partición. Útil si $M \gg \sqrt{b_s \times b_b}$.

15.5.6 Resumen comparativo de juntas

- **Nested loops:** cualquier predicado de junta. Mejor si outer es muy chica.
- **Merge join:** solo equi-joins. Óptimo si ambas relaciones ya ordenadas.
- **Hash join:** solo equi-joins. Óptimo para relaciones grandes sin índice.
- **Indexed nested loops:** ideal si existe índice en inner y outer es chica.

15.6 — Otras Operaciones

15.6.1 Eliminación de duplicados

Dos métodos: **sorting** (ordenar, eliminar adyacentes iguales) o **hashing** (particionar, construir hash index in-memory descartando duplicados). Costo similar al de sort/hash join sobre la build relation.

SQL requiere DISTINCT explícito; sino, duplicados se retienen.

15.6.2 Proyección

Proyectar cada tupla y luego eliminar duplicados (si es necesario). Si la proyección incluye una clave, no hay duplicados.

15.6.3 Operaciones de conjunto ($\cup$, $\cap$, $-$)

Por sorting: ordenar ambas relaciones, recorrer en paralelo.

- $r \cup s$: retener una copia de cada tupla.
- $r \cap s$: retener solo tuplas presentes en ambas.
- $r - s$: retener tuplas de r ausentes en s .

Costo: $b_r + b_s$ transferencias + seeks (si ya ordenadas).

Por hashing: particionar ambas con misma función hash, luego operar por partición con hash index in-memory.

15.6.4 Outer Join

Dos estrategias:

1. Computar el join normal, luego agregar tuplas sin match con NULLs.
2. Modificar algoritmos de join (nested loops, merge, hash) para emitir tuplas sin match con NULLs directamente.

15.6.5 Agregación

GROUP BY + funciones de agregación (SUM, AVG, MIN, MAX, COUNT): implementar igual que eliminación de duplicados, pero agrupando y aplicando funciones. Puede hacerse *on-the-fly*: mantener acumuladores parciales mientras se construyen los grupos.

Si resultado cabe en memoria: b_r transferencias y 1 seek (sin escribir a disco).

15.7 — Evaluación de Expresiones

15.7.1 Materialización

Evaluar una operación a la vez, de abajo hacia arriba en el **árbol de operadores**. El resultado de cada operación intermedia se **materializa** (escribe en una relación temporal).

Costo total = suma de costos individuales + costo de escribir/leer resultados intermedios. Los resultados intermedios pueden ser grandes → costoso.

Double buffering: usar dos buffers de salida para solapar CPU con I/O.

15.7.2 Pipelining

Combinar varias operaciones en un **pipeline**: el resultado de una operación se pasa **directamente** a la siguiente sin materializar.

Beneficios:

1. Elimina costo de leer/escribir relaciones temporales.
2. Resultados parciales disponibles más rápido.

Implementación:

- **Demand-driven (lazy, pulling):** cada operador implementa un **iterador** con funciones `open()`, `next()`, `close()`. El operador raíz “tira” tuplas de sus hijos. Más común.
- **Producer-driven (eager, pushing):** cada operador genera tuplas activamente y las “empuja” hacia su padre usando buffers intermedios. Útil en sistemas paralelos.

Operaciones bloqueantes vs. pipelineables

- **Operaciones bloqueantes:** necesitan ver toda la entrada antes de producir salida. Ej: sort (la fase de generación de runs acepta pipeline, pero la fase de merge necesita que los runs estén completos).
- **Hash join:** bloqueante (necesita particionar ambas entradas antes del build-probe). Pero **hybrid hash join** puede hacer pipeline parcial si la primera partición de build cabe en memoria.
- **Indexed nested-loop join:** pipeline en la entrada outer, bloqueante en inner (necesita el índice).

Un **pipeline stage** es un subárbol del plan donde todas las aristas son pipelined. Entre stages, las aristas son **materialized edges (blocking edges)**.

Capítulo 16 — Optimización de Consultas (16.1–16.4)

16.1 — Panorama General

La **optimización de consultas** es el proceso de seleccionar el plan de evaluación más eficiente. No se espera que el usuario escriba consultas eficientes; el sistema debe construir un plan que minimice el costo.

Tres pasos:

1. Generar expresiones equivalentes mediante **reglas de equivalencia**.
2. Anotar cada expresión con algoritmos alternativos para generar **planes de evaluación**.
3. Estimar el costo de cada plan y elegir el de menor costo estimado.

Ver planes de ejecución:

- PostgreSQL: EXPLAIN <query>
- SQL Server: SET SHOWPLAN_TEXT ON
- Oracle: EXPLAIN PLAN FOR
- MySQL: EXPLAIN <query> + SHOW WARNINGS

16.2 — Transformación de Expresiones Relacionales

Dos expresiones de álgebra relacional son **equivalentes** si producen el mismo conjunto de tuplas en toda instancia legal de la BD.

16.2.1 Reglas de Equivalencia

1. **Cascada de σ** : $\sigma_{\theta_1 \wedge \theta_2}(E) \equiv \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
2. **Conmutatividad de σ** : $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) \equiv \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
3. **Cascada de Π** : $\Pi_{L_1}(\Pi_{L_2}(\dots(\Pi_{L_n}(E))\dots)) \equiv \Pi_{L_1}(E)$ si $L_1 \subseteq L_2 \subseteq \dots$
4. **σ con $\times$ y $\bowtie$** :
 - $\sigma_{\theta}(E_1 \times E_2) \equiv E_1 \bowtie_{\theta} E_2$
 - $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) \equiv E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$
5. **Conmutatividad de $\bowtie$** : $E_1 \bowtie_{\theta} E_2 \equiv E_2 \bowtie_{\theta} E_1$
6. **Asociatividad de $\bowtie$** : $(E_1 \bowtie E_2) \bowtie E_3 \equiv E_1 \bowtie (E_2 \bowtie E_3)$
7. **Distribución de σ sobre $\bowtie$** :
 - Si θ_1 solo involucra atributos de E_1 : $\sigma_{\theta_1}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} E_2$
 - Si θ_1 involucra E_1 y θ_2 involucra E_2 : $\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) \equiv (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$
8. **Distribución de Π sobre $\bowtie$** : se puede proyectar antes del join si se retienen los atributos necesarios para la condición de junta.
9. **Conmutatividad de $\cup$ y $\cap$** (diferencia **no** es conmutativa).
10. **Asociatividad de $\cup$ y $\cap$** .
11. **Distribución de σ sobre $\cup, \cap, -$** .
12. **Distribución de Π sobre $\cup$** .

Reglas adicionales para outer joins:

- Full outer join es conmutativo; left/right no (pero $E_1 \text{ LOJ } E_2 \equiv E_2 \text{ ROJ } E_1$).
- Outer joins **no** son asociativos.
- Outer join $\rightarrow$ inner join si la selección es **null rejecting** (rechaza NULLs de la tabla preservada).

16.2.2 Ejemplos de Transformaciones

Aplicando regla 7.a (*push selection*):

$$\sigma_{\text{dept}=\text{"Music"}}(\text{instructor} \bowtie \text{teaches}) \equiv (\sigma_{\text{dept}=\text{"Music"}}(\text{instructor})) \bowtie \text{teaches}$$

Reduce el tamaño del resultado intermedio, bajando el costo.

Push projections: eliminar atributos innecesarios antes de un join reduce el tamaño de los resultados intermedios.

16.2.3 Orden de Juntas (Join Ordering)

El natural join es **conmutativo** y **asociativo**: $(r_1 \bowtie r_2) \bowtie r_3 \equiv r_1 \bowtie (r_2 \bowtie r_3)$, pero los costos pueden ser muy diferentes. El optimizador debe elegir el orden que minimiza los resultados intermedios.

Con n relaciones, hay $\frac{(2(n-1))!}{(n-1)!}$ órdenes de junta posibles. Para $n = 3$: 12; $n = 7$: 665,280; $n = 10$: > 17,6 mil millones.

16.2.4 Enumeración de Expresiones Equivalentes

Se generan sistemáticamente aplicando reglas de equivalencia. Optimizaciones: (1) compartir subexpresiones entre expresiones equivalentes, (2) podar alternativas que ya superan el mejor costo conocido.

16.3 — Estimación de Estadísticas de Resultados

16.3.1 Información del Catálogo

Estadísticas almacenadas para cada relación r :

- n_r : número de tuplas.
- b_r : número de bloques. $b_r = \lceil n_r / f_r \rceil$.
- l_r : tamaño de una tupla en bytes.
- f_r : **factor de bloqueo** (tuplas por bloque).
- $V(A, r)$: número de valores distintos del atributo A en r .

También: alturas de B⁺-trees, número de hojas. Las estadísticas se actualizan periódicamente (no en cada modificación). Comando: ANALYZE (PostgreSQL/Oracle), RUNSTATS (DB2).

Histogramas: distribución de valores de un atributo.

- **Equi-width:** rangos de igual tamaño.
- **Equi-depth:** rangos con igual cantidad de tuplas ($\rightarrow$ mejores estimaciones).

Muchas BDs también almacenan los n valores más frecuentes con sus conteos.

16.3.2 Estimación de Tamaño de Selecciones

- $\sigma_{A=a}(r)$: estimado = $n_r / V(A, r)$ tuplas (asumiendo distribución uniforme). La probabilidad s_i / n_r se llama **selectividad**.
- $\sigma_{A \leq v}(r)$: estimado = $n_r \cdot \frac{v - \min(A, r)}{\max(A, r) - \min(A, r)}$. Si v desconocido: $n_r / 2$.
- **Conjunción** $\sigma_{\theta_1 \wedge \dots \wedge \theta_n}(r)$: asumiendo independencia, $n_r \cdot \frac{s_1 \cdot s_2 \cdot \dots \cdot s_n}{n_r^n}$.
- **Disyunción** $\sigma_{\theta_1 \vee \dots \vee \theta_n}(r)$: $n_r \cdot \left(1 - \prod_{i=1}^n \left(1 - \frac{s_i}{n_r}\right)\right)$.
- **Negación** $\sigma_{\neg \theta}(r)$: $n_r - s_\theta$.

16.3.3 Estimación de Tamaño de Juntas

- Si $R \cap S = \emptyset$: $|r \bowtie s| = n_r \times n_s$ (producto cartesiano).
- Si $R \cap S$ es clave de R : $|r \bowtie s| \leq n_s$. Si además es FK de S referenciando R : $|r \bowtie s| = n_s$.
- Caso general ($R \cap S = \{A\}$): estimado = $\frac{n_r \times n_s}{\max(V(A,r), V(A,s))}$ (se elige el menor de las dos estimaciones simétricas).

16.3.4 Estimación de Tamaño de Otras Operaciones

- **Proyección:** $|\Pi_A(r)| = V(A, r)$.
- **Agregación:** $|G_{\mathcal{Y}_A}(r)| = V(G, r)$.
- **Operaciones de conjunto:** $|r \cup s| \leq n_r + n_s$; $|r \cap s| \leq \min(n_r, n_s)$; $|r - s| \approx n_r$.
- **Outer join:** $|r \text{ LOJ } s| = |r \bowtie s| + n_r$; $|r \text{ FOJ } s| = |r \bowtie s| + n_r + n_s$ (cotas superiores).

16.3.5 Estimación de Valores Distintos

Para selecciones $V(A, \sigma_\theta(r))$:

- θ fuerza $A = a$: $V(A, \sigma_\theta(r)) = 1$.
- θ es comparación: $V(A, \sigma_\theta(r)) \approx V(A, r) \times s$ (s = selectividad).
- Otros casos: $\min(V(A, r), n_{\sigma_\theta(r)})$.

Para juntas $V(A, r \bowtie s)$: si A viene solo de r , $\min(V(A, r), n_{r \bowtie s})$. Si A viene de ambos: $\min(V(A_1, r) \times V(A_2, s), n_{r \bowtie s})$.

16.4 — Elección de Planes de Evaluación

16.4.1 Selección de Orden de Juntas Basada en Costo

Un **optimizador basado en costo** explora el espacio de planes equivalentes y elige el de menor costo estimado.

Algoritmo de programación dinámica (FindBestPlan):

1. Para un conjunto S de relaciones, si el mejor plan ya fue computado, retornarlo (*memoization*).
2. Si $|S| = 1$: elegir el mejor método de acceso (scan, index scan).
3. Si $|S| > 1$: probar toda partición de S en dos subconjuntos S_1 y $S - S_1$. Para cada partición, recursivamente encontrar los mejores planes de S_1 y $S - S_1$, y probar todos los algoritmos de junta.
4. Guardar el plan de menor costo en **bestplan**[S].

Complejidad: $O(3^n)$ para n relaciones. Mucho mejor que $O((2n)!/n!)$ de fuerza bruta.

Interesting sort orders: un plan que genera resultados ordenados por un atributo útil para operaciones posteriores puede ser preferible aunque sea más caro localmente. **bestplan** se indexa por $[S, o]$ donde o es un orden de interés.

16.4.2 Optimización Basada en Reglas de Equivalencia

Para manejar outer joins, agregaciones, subqueries y otras operaciones más allá de inner joins, se usan **reglas de equivalencia físicas** que transforman operaciones lógicas en físicas (ej: join $\rightarrow$ hash join, merge join, nested loops).

Técnicas clave:

1. Representación eficiente que comparte subexpresiones.

2. Detección de derivaciones duplicadas.
3. **Memoization**: guardar el plan óptimo por subexpresión.
4. Poda: descartar planes cuyo costo parcial ya excede el mejor conocido.

Este enfoque fue pionero del proyecto **Volcano**; el optimizador de SQL Server se basa en él.

16.4.3 Heurísticas en Optimización

- **Realizar selecciones lo antes posible** (*push selections early*): reduce el tamaño de resultados intermedios. Generalmente beneficioso, aunque no siempre óptimo.
- **Realizar proyecciones lo antes posible** (*push projections early*): eliminar atributos innecesarios.
- **Left-deep join trees**: muchos optimizadores (ej: System R) solo consideran órdenes de junta donde el operando derecho es siempre una relación base. Facilita pipelining (solo un input por join necesita ser materializado). Complejidad: $O(n \cdot 2^n)$ vs. $O(3^n)$ general.
- **Presupuesto de optimización**: si un plan barato se encuentra rápido, se reduce el presupuesto de búsqueda. Si el mejor plan es caro, se invierte más tiempo optimizando.
- **Plan caching**: reusar planes de consultas parametrizadas ejecutadas previamente.

16.4.4 Optimización de Subqueries Anidadas

SQL trata subqueries del **WHERE** como funciones que toman parámetros (**correlation variables**) del query externo.

Evaluación correlacionada: ejecutar el subquery **por cada tupla** del query externo → muy costoso (muchos accesos aleatorios).

Decorrelación: transformar subqueries en joins o semijoin/anti-semijoin:

- **EXISTS** → **semijoin** ($\bowtie$): retiene tuplas de r que tienen al menos un match en s , preservando duplicados de r .
- **NOT EXISTS** → **anti-semijoin** ($\bar{\bowtie}$): retiene tuplas de r que **no** tienen match en s .
- **IN** se traduce igual que **EXISTS**.

La decorrelación permite usar algoritmos de junta eficientes (hash, merge) en lugar de evaluación correlacionada.